

Interview Questions — Django REST Framework (REST API Development)

Total Questions: 47

1. What is an API and why are APIs important in web development?

Hint: Think about how different systems communicate with each other.

2. Explain what a REST API is and name two key principles it follows.

Hint: Consider statelessness and how resources are handled.

3. What is the difference between JSON and XML when it comes to data exchange in APIs?

Hint: Focus on readability and typical use in modern APIs.

4. How is Django REST Framework (DRF) different from standard Django?

Hint: Think about what DRF adds to Django for API development.

5. Describe the steps to install Django REST Framework in a Django project.

Hint: Consider both package installation and settings configuration.

6. What is the purpose of adding 'rest_framework' to INSTALLED_APPS in Django settings?

7. What is the difference between a Serializer and a ModelSerializer in DRF?

Hint: Think about which one is more automatic and when you'd use each.

8. How would you create a simple 'Hello API' endpoint in DRF using APIView?

9. What is a Django model, and why do we define models before creating API endpoints?

Hint: Consider how models relate to the database.

10. Write a basic DRF Serializer for a model called Doctor with fields 'name' and 'specialization'.

Hint: Remember to import Serializer and define fields.

11. How do you create a CRUD API endpoint for a model using APIView in DRF?

Hint: Think about the HTTP methods involved and their mapping.

12. What is the difference between HTTP status codes 200 and 201 in API responses?

Hint: One is for successful retrieval, the other for successful creation.

13. How would you return a JSON response from a DRF API endpoint?

14. Write the code for a POST endpoint to create a new Doctor using ModelSerializer in DRF.

15. What is the use of GenericAPIView and Mixins in DRF?

Hint: Think about reducing code repetition for common operations.

16. Describe how you would update a Doctor object using a PUT request in DRF.

17. Explain what a ViewSet is in DRF and how it differs from APIView.

Hint: Focus on how ViewSets group logic and reduce code.

18. What is the purpose of a Router in DRF, and what does it generate automatically?

Hint: Think about URL patterns for your API.

19. Explain the difference between DefaultRouter and SimpleRouter in DRF.

20. How does pagination help when returning large querysets from an API?

Hint: Consider both server performance and client usability.

21. Describe how you would implement PageNumberPagination for a list endpoint in DRF.

22. What is the difference between LimitOffsetPagination and CursorPagination in DRF?

23. How would you add ordering by 'name' to a list endpoint in DRF?

24. What are the main types of authentication available in DRF?

Hint: List at least three and describe them briefly.

25. Explain the difference between TokenAuthentication and SessionAuthentication in DRF.

Hint: Consider where the authentication information is stored.

26. Write a DRF permission class that only allows access to authenticated users.

27. How would you protect the `/api/doctors/` endpoint so that only logged-in users can access it?

28. What is the role of the `IsAdminUser` permission class in DRF?

29. If you wanted to create a custom permission that only allows doctors with a certain specialization to edit their profile, how would you approach this in DRF?

Hint: Think about overriding the `'has_permission'` or `'has_object_permission'` methods.

30. How would you generate and use an authentication token for a user in DRF?

31. What is JWT and how is it different from basic token authentication?

Hint: Think about the structure and usage of JWTs.

32. How could you use ChatGPT or Copilot to help you explain the difference between Token and Session Authentication in an interview? What would you check before trusting the output?

Hint: Think about verifying explanations and cross-checking with official docs.

33. Describe the steps to integrate an external API, such as OpenWeatherMap, into a DRF endpoint.

34. Write a DRF view that calls an external weather API and returns the temperature for a given city.

35. How would you handle errors if the external API you called from your DRF endpoint is down or returns invalid data?

Hint: Consider exception handling and response status codes.

36. What is Postman and how can it help you test your DRF API endpoints?

Hint: Focus on features that make API testing easier.

37. How could you use Postman AI or ChatGPT to generate code snippets for calling your DRF API from Python?

Hint: Think about the `'Code'` feature and checking if the code matches your API's requirements.

-
38. Explain how you would send an email using an external service like Mailgun or MailChimp from a DRF endpoint.
-
39. Describe how you would integrate Twilio to send an SMS notification from your DRF API.
-
40. What steps are involved in deploying a DRF API to PythonAnywhere?
- Hint:** Think about preparing your project, setting up the web app, and configuring settings.
-
41. Why is API versioning important, and how would you implement it in DRF?
- Hint:** Consider backward compatibility and URL structure.
-
42. How would you format a custom JSON response in DRF, for example, to wrap all data in a 'result' key?
-
43. Describe how exception handling in DRF can improve the robustness of your API.
-
44. Imagine you have deployed your DRF API and a client reports that the `/api/doctors/` endpoint returns a 500 error. How would you troubleshoot this?
- Hint:** Think about checking logs, error messages, and recent code changes.
-
45. Integrative: How would you secure an endpoint that integrates with an external payment API, ensuring only authenticated users can access it and handling errors gracefully?
- Hint:** Combine authentication, permissions, and exception handling.
-
46. Integrative: Describe how you would create a paginated, authenticated endpoint that lists doctors and allows filtering by specialization.
-
47. Integrative: Suppose you need to update your API to v2, adding a new required field to the Doctor model. How would you handle this change to avoid breaking existing clients?
- Hint:** Think about versioning and backward compatibility.
-